Sunthareswaran Sharujan

TD04 : Projet MarketPulse

Branches et merge : isoler une fonctionnalité avant de l'intégrer

Module MESIFI472326 – Python, Git, Linux · Groupe G07 · Équipe TSHA



Enseignant : ELANGA NKO'O Steve Patrick Lionel **Année Universitaire :2026/2027**

Table des matières

1. Introduction **3**

2. TD04 – Branches + Merge **3**

2.1. Partie 1 – Principe des branches 3
2.2. Partie 2 – Vérifier la branche courante 3
2.3. Partie 3 – Créer la branche 3
2.4. Partie 4 – La fonctionnalité 3
2.5. Partie 5 – Commit sur la branche 4
2.6. Partie 6 – Comparer avec main 5
2.7. Partie 7 – Merge dans main 6
2.8. Partie 8 – Capture du Checkpoint et suppression de la branche 6
2.9. Partie 9 – Les conflits de merge 7
2.10. Partie 10 – Validation 7
2.11. Checkpoint A 7

3. Conclusion **7**

1. Introduction

Au TD03, mon travail du TD02 a été enregistré dans un commit sur `main`. Le TD04 ajoute une nouvelle question : comment développer une fonctionnalité sans toucher à `main` tant qu'elle n'est pas prête ? La réponse est d'utiliser une branche. La fonctionnalité à ajouter est l'affichage de la première et de la dernière date d'observation pour l'instrument et le benchmark. Le TD se termine par le Checkpoint A.

2. TD04 – Branches + Merge

2.1. Partie 1 – Principe des branches

Une branche permet de travailler sur une fonctionnalité à part. Tant qu'on ne fait pas de merge, `main` ne change pas : si la fonctionnalité ne marche pas ou est abandonnée, la version principale n'est pas touchée. Quand le travail est prêt, on le fusionne dans `main`. À partir de ce TD, on ne développe plus directement sur `main`.

2.2. Partie 2 – Vérifier la branche courante

Avant de créer la branche, il faut être sur `main` avec un dépôt propre. J'avais encore une ligne de test oubliée du TD03 dans `main.py`, que j'ai annulée avec `git restore --staged src/main.py` puis `git restore src/main.py`.

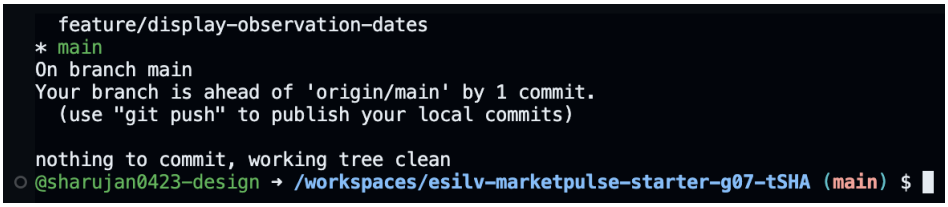


Figure 1 – git branch et git status sur main.

L'étoile de `git branch` indique la branche courante (`main`) et `git status` confirme qu'il n'y a rien en attente. La capture a été prise après la création de la branche, mais `main` n'avait pas bougé entre-temps.

2.3. Partie 3 – Créer la branche

J'ai créé la branche avec `git switch -c feature/display-observation-dates`. L'option `-c` crée la branche et bascule dessus directement.



Figure 2 – git branch : je suis sur la nouvelle branche.

Le nom suit la convention de l'énoncé : `feature/` pour une nouvelle fonctionnalité, puis une description du travail. On évite les noms vagues comme `test` ou `mybranch`.

2.4. Partie 4 – La fonctionnalité

J'ai ajouté deux fonctions sur le modèle de `get_first_close` et `get_last_close` :

```python
def get_first_date(prices):
    return prices[0]["date"]

def get_last_date(prices):
    return prices[-1]["date"]
```
Python

Puis deux lignes dans `display_market_summary`, juste après le nombre d'observations. Comme cette fonction sert pour les deux séries, les dates s'affichent pour AAPL et pour SP500 sans rien écrire de plus.



Figure 3 – python src/main.py sur la branche : les dates s'affichent.

Les deux séries vont bien du 2026-09-01 au 2026-09-30. Les deux dernières lignes du S&P 500 ont été coupées par l'affichage du terminal Codespaces, mais elles ne concernent pas cette partie.

2.5. Partie 5 – Commit sur la branche

```
$ git --no-pager diff --stat
 src/main.py | 10 ++++++++++
 1 file changed, 10 insertions(+)

$ git add src/main.py
$ git commit -m "feat: display first and last observation dates"
[feature/display-observation-dates 6aef068] feat: display first and last
observation dates
 1 file changed, 10 insertions(+)
```
Terminal

Le diff ne contient que des ajouts (10 lignes). Git indique entre crochets que le commit est enregistré sur la branche et non sur `main`.

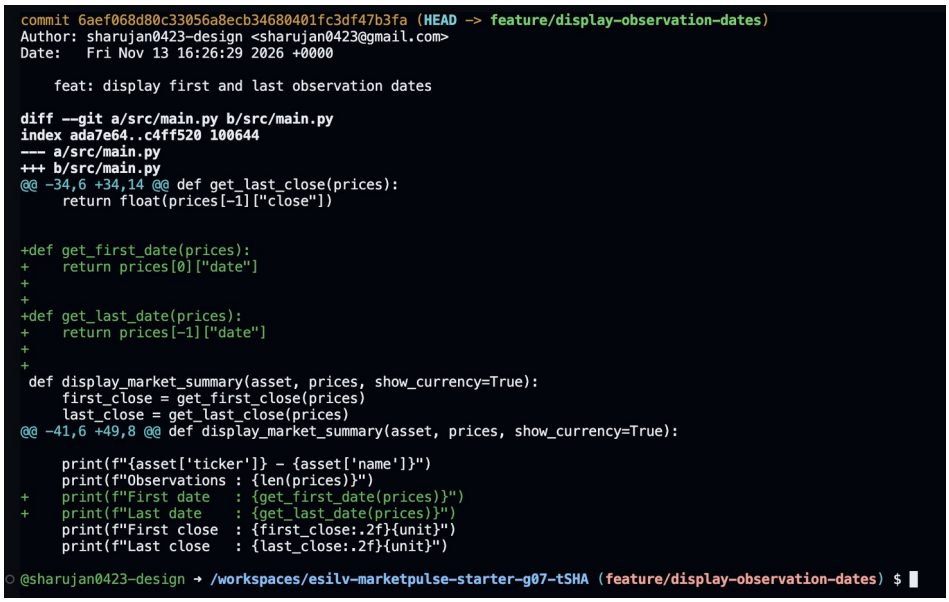


Figure 4 – git show HEAD : le commit 6aef068 sur la branche.

2.6. Partie 6 – Comparer avec main

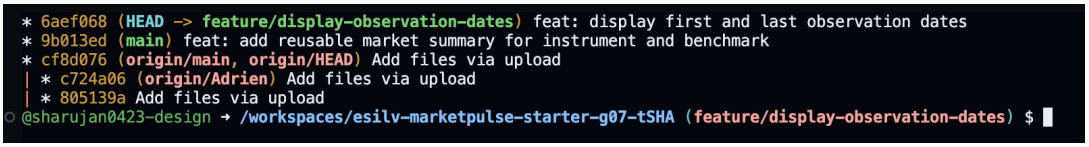


Figure 5 – git log --oneline --graph --decorate --all.



Figure 6 – git diff main..feature/display-observation-dates.

- Changement présent sur la branche : l'affichage des dates (les deux fonctions et les deux print).
- Il n'est pas encore dans `main` car le commit a été fait sur la branche : `main` est restée sur `9b013ed`.
- Si la branche était abandonnée avant le merge, `main` resterait comme avant, sans les dates.

On voit aussi `origin/Adrien`, la branche d'un coéquipier sur GitHub, car l'option `--all` affiche toutes les branches.

2.7. Partie 7 – Merge dans main



Figure 7 – Retour sur main.

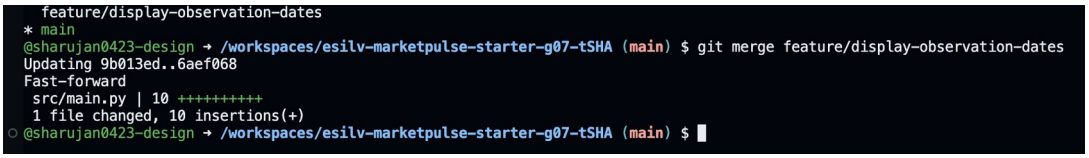


Figure 8 – git merge feature/display-observation-dates.

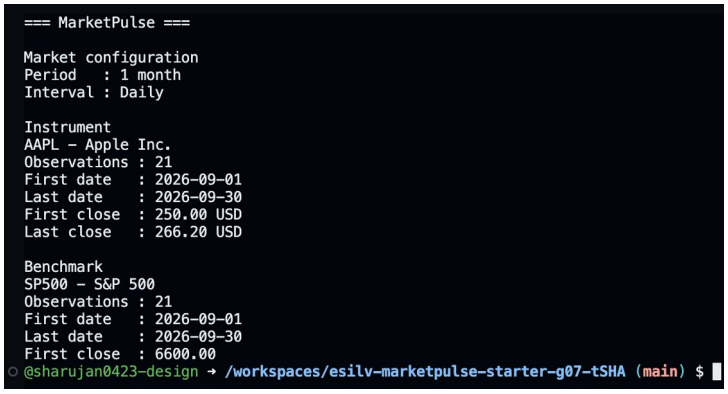


Figure 9 – python src/main.py depuis main après le merge.

- La fonctionnalité est maintenant visible depuis `main` : les dates s'affichent.
- Le commit `6aef068` est toujours dans l'historique, il fait maintenant partie de `main`.
- Git a fait un fast-forward : `main` n'avait pas reçu de nouveau commit depuis la création de la branche, donc il a simplement avancé `main` jusqu'au commit de la branche, sans créer de commit de merge.
- Le graphe reste donc linéaire, ce qui est valide d'après l'énoncé.

2.8. Partie 8 – Capture du Checkpoint et suppression de la branche

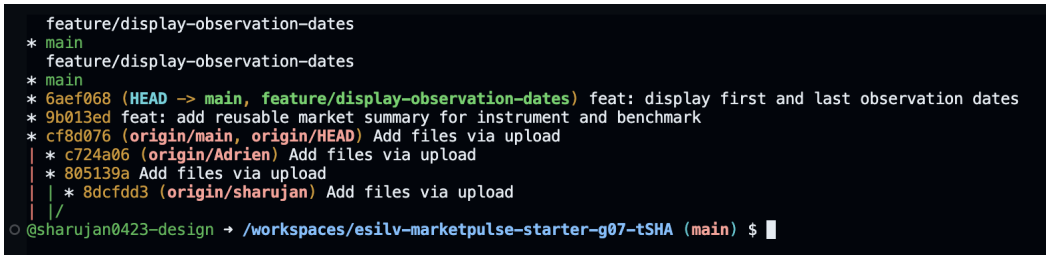


Figure 10 – git branch, git branch --merged et le graphe (03_branch_merge.png).

La branche apparaît dans `git branch --merged`, ce qui prouve qu'elle est intégrée, et `main` et la branche pointent sur le même commit `6aef068`. J'ai pris cette capture avant de supprimer la branche avec `git branch -d feature/display-observation-dates`. L'option `-d` refuse de supprimer une branche qui n'est pas fusionnée, contrairement à `-D`.

2.9. Partie 9 – Les conflits de merge

Un conflit arrive quand deux branches modifient les mêmes lignes d'un fichier : Git ne peut pas choisir seul quelle version garder. Il ajoute alors des marqueurs `<<<<<<<`, `=======` et `>>>>>>>` dans le fichier et c'est à nous de choisir le contenu final, de retirer les marqueurs, puis de faire `git add` et de terminer le merge. Ce n'est pas une erreur de Git, juste une décision à prendre. Ici, il n'y a pas eu de conflit puisque `main` n'avait pas bougé.

2.10. Partie 10 – Validation

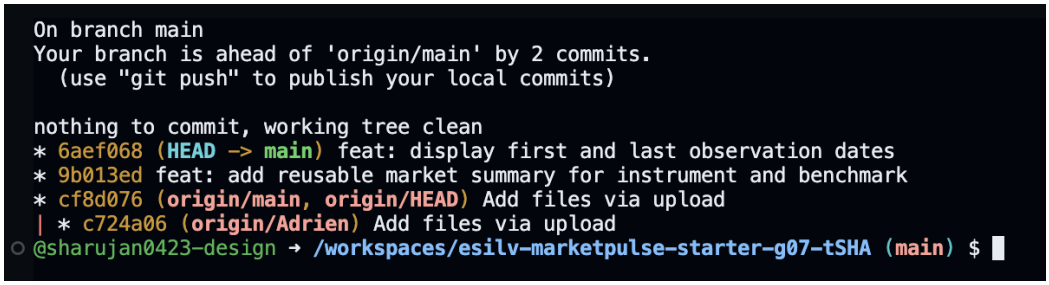


Figure 11 – État final : git status et historique.

Le dépôt est propre et `main` contient mes deux commits (TD03 et TD04), qui ne sont qu'en local (« ahead of 'origin/main' by 2 commits »). Le nom de la branche a disparu du graphe, mais son commit reste dans l'historique. MarketPulse fonctionne après le merge. Le CORE du TD04 est validé.

2.11. Checkpoint A

Les preuves sont prêtes dans le dossier `evidence/checkpoint-a/sharujan0423-design/` : `README.md`, `01_terminal_python.png`, `02_git_status_log.png` (TD03) et `03_branch_merge.png` (figure 10).

3. Conclusion

Ce TD m'a montré l'intérêt des branches : j'ai pu développer et tester l'affichage des dates sans toucher à `main`, puis l'intégrer d'un coup avec un merge. Le fast-forward était le cas le plus simple, mais j'ai compris ce qui se passerait si `main` avait aussi changé : un vrai commit de merge, voire un conflit à résoudre. Au TD05, ces branches seront envoyées sur GitHub et passeront par une Pull Request avant d'être fusionnées.